01. Business Problem & Objective

Frame recommendations as a ranking problem, not a classification problem.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

Business problem

A movie catalogue contains thousands of titles. A user who likes one film may want to discover other films with similar themes, genres, keywords, cast or director. The system should return a short ranked list of relevant movies for a selected title.

Problem framing

This repository implements a **content-based recommender**. It represents each movie through descriptive metadata and text, then compares movies using cosine similarity. It is not collaborative filtering: it does not learn from user-item ratings or watch histories.

- Input: a movie title selected from the catalogue.
- Output: five other movie titles with the highest content-vector similarity.
- Core objective: rank items whose metadata representation is close to the selected movie.
- No explicit user profile, ratings, watch events, or relevance labels are used.

Business success criteria

A useful recommender should return relevant and diverse suggestions, cover enough of the catalogue, respond quickly, and avoid repeatedly recommending near-duplicates. Evaluation should include offline ranking metrics and qualitative human review. Cosine similarity alone measures vector closeness, not whether users actually enjoy the recommendations.

Project scope and repository behaviour

- Loads TMDB 5000 movies and credits CSV files in the notebook; these CSV files are referenced but are not included in the supplied ZIP.
- Merges movie and credit metadata by title, selects movie_id/title/overview/genres/keywords/cast/crew.
- Extracts names from JSON-like metadata, retains top three cast members, extracts directors, combines text into tags.
- Uses CountVectorizer(max_features=5000, stop_words='english') and cosine_similarity.
- Saves movie_list.pkl and similarity.pkl. The Streamlit app loads these pickles, takes a selected title and shows five recommendations with posters fetched from TMDB.

Interview framing

> “I built a content-based movie recommender. Instead of relying on other users’ ratings, I create a text representation of each movie using its overview, genres, keywords, top cast and director. I convert those tags into a bag-of-words vector and rank other movies by cosine similarity to the selected title.”